DATABASE TECHNOLOGIES LAB SHEET – 4

Course Code: MCA105P

SECTION 1: DDL COMMANDS — SCHOOLDB

Q1. Create a database named SchoolDB.

```
CREATE DATABASE SchoolDB;
```

Q2. Create a Students table with StudentID, FirstName, LastName, DateOfBirth and Gender.

```
USE SchoolDB;

CREATE TABLE Students (
   StudentID INT PRIMARY KEY,
   FirstName VARCHAR(50),
   LastName VARCHAR(50),
   DateOfBirth DATE,
   Gender CHAR(1)
);
```

Q3. Drop the Students table from SchoolDB.

```
DROP TABLE Students;
```

Q4. Create the Students table again as defined in Question 2.

```
CREATE TABLE Students (
   StudentID INT PRIMARY KEY,
   FirstName VARCHAR(50),
   LastName VARCHAR(50),
   DateOfBirth DATE,
   Gender CHAR(1)
);
```

Q5. Add a new Email column to the Students table.

```
ALTER TABLE Students
ADD Email VARCHAR(100);
```

Q6. Modify the LastName column to allow NULL values.

```
ALTER TABLE Students
MODIFY LastName VARCHAR(50) NULL;
```

Q7. Change the DateOfBirth column data type to DATETIME.

```
ALTER TABLE Students
MODIFY DateOfBirth DATETIME;
```

Q8. Insert some sample data into the Students table.

```
INSERT INTO Students
(StudentID, FirstName, LastName, DateOfBirth, Gender, Email)
VALUES
(1, 'Rahul', 'Sharma', '2004-05-15', 'M', 'rahul@example.com'),
(2, 'Priya', 'Verma', '2005-08-20', 'F', 'priya@example.com'),
(3, 'Amit', 'Kumar', '2004-11-10', 'M', 'amit@example.com');

SELECT * FROM Students;
```

Q9. Truncate the Students table to remove all records.

```
TRUNCATE TABLE Students;
```

Q10. Rename the Students table to Pupils.

```
RENAME TABLE Students TO Pupils;
```

Q11. Rename the Email column to ContactEmail in the Pupils table.

```
ALTER TABLE Pupils
RENAME COLUMN Email TO ContactEmail;
```

SECTION 2: CREATE TABLES FOR A LIBRARY

Q12. Create a Books table with the required columns.

```
CREATE TABLE Books (
   book_id INT PRIMARY KEY,
   title VARCHAR(255) NOT NULL,
   author VARCHAR(255),
   isbn VARCHAR(13) UNIQUE,
   genre VARCHAR(50)
);
```

Q13. Create a Members table with the required columns.

```
CREATE TABLE Members (
   member_id INT PRIMARY KEY,
   name VARCHAR(255) NOT NULL,
   email VARCHAR(255) UNIQUE,
   phone_number VARCHAR(20)
);
```

Q14. Create a Loans table with foreign keys referencing Books and Members.

```
CREATE TABLE Loans (
   loan_id INT PRIMARY KEY,
   book_id INT,
   member_id INT,
```

```
    loan_date DATE,
    return_date DATE NULL,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
```

SECTION 3: LAB QUESTIONS — CONSTRAINTS AND INDEXES

Note: The following questions use the Student, Department, Course and Enrollment tables from the lab sheet.

Q15. Create a Department table where DeptName cannot be NULL. Insert at least 3 records.

```
CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(100) NOT NULL,
    Location VARCHAR(50)
);

INSERT INTO Department (DeptID, DeptName, Location)
VALUES
(1, 'Computer Science', 'Block A'),
(2, 'Mathematics', 'Block B'),
(3, 'Physics', 'Block C');

SELECT * FROM Department;
```

Q16. Insert a new record into the Student table without providing Name. What happens?

```
INSERT INTO Student (StudentID, Email, Age, DeptID)
VALUES (105, 'test@example.com', 20, 1);
```

Q17. Modify the Student table so that Email must be unique. Try inserting a duplicate email.

```
ALTER TABLE Student
ADD CONSTRAINT unique_student_email UNIQUE (Email);

INSERT INTO Student (StudentID, Name, Email, Age, DeptID)
VALUES (105, 'Karan', 'rahul@example.com', 20, 1);.
```

Q18. Create a Library table where BookID is unique for every record. Insert 5 books.

```
CREATE TABLE Library (
    BookID INT UNIQUE,
    BookName VARCHAR(100),
    Author VARCHAR(100)
);
```

```
INSERT INTO Library (BookID, BookName, Author)
VALUES
(1, 'Database Systems', 'Ramakrishnan'),
(2, 'Python Basics', 'Guido van Rossum'),
(3, 'Java Programming', 'Herbert Schildt'),
(4, 'Operating Systems', 'Galvin'),
(5, 'Computer Networks', 'Tanenbaum');

SELECT * FROM Library;
```

Q19. Define StudentID in the Student table as a primary key. Try inserting a student with the same StudentID.

```
ALTER TABLE Student
ADD PRIMARY KEY (StudentID);

INSERT INTO Student (StudentID, Name, Email, Age, DeptID)
VALUES (101, 'Karan', 'karan@example.com', 20, 1);
```

Q20. Create a CourseInstructor table with InstructorID as the primary key. Insert at least 3 records.

```
CREATE TABLE CourseInstructor (
   InstructorID INT PRIMARY KEY,
   InstructorName VARCHAR(100),
   CourseName VARCHAR(100)
);

INSERT INTO CourseInstructor
(InstructorID, InstructorName, CourseName)
VALUES
(1, 'Amit Sharma', 'Database Systems'),
(2, 'Neha Verma', 'Mathematics'),
(3, 'Rahul Singh', 'Computer Networks');

SELECT * FROM CourseInstructor;
```

Q21. Add a foreign key constraint between Student(DeptID) and Department(DeptID).

```
ALTER TABLE Student
ADD CONSTRAINT fk_student_department
FOREIGN KEY (DeptID)
REFERENCES Department(DeptID);
```

Q22. Add a foreign key constraint between Enrollment(StudentID) and Student(StudentID).

```
ALTER TABLE Enrollment
ADD CONSTRAINT fk_enrollment_student
FOREIGN KEY (StudentID)
REFERENCES Student(StudentID);
```

Q23. Create a CHECK constraint on Student so that Age must be greater than 18. Test it with an invalid record.

```
ALTER TABLE Student
ADD CONSTRAINT chk_student_age CHECK (Age > 18);

INSERT INTO Student (StudentID, Name, Email, Age, DeptID)
VALUES (106, 'Ravi', 'ravi@example.com', 16, 1);
```

Q24. Add a CHECK constraint on Course so that Credits must be between 1 and 6.

```
ALTER TABLE Course
ADD CONSTRAINT chk_course_credits
CHECK (Credits BETWEEN 1 AND 6);
```

Q25. Add a JoiningDate column to Student with the current date as its default value. Insert a student without specifying JoiningDate.

```
ALTER TABLE Student
ADD JoiningDate DATE DEFAULT (CURRENT_DATE);

INSERT INTO Student (StudentID, Name, Email, Age, DeptID)
VALUES (107, 'Pooja', 'pooja@example.com', 21, 2);

SELECT * FROM Student;
```

Q26. Create a Faculty table where Designation has the default value Lecturer. Insert a record without specifying Designation.

```
CREATE TABLE Faculty (
   FacultyID INT PRIMARY KEY,
   FacultyName VARCHAR(100),
   Designation VARCHAR(50) DEFAULT 'Lecturer'
);

INSERT INTO Faculty (FacultyID, FacultyName)
VALUES (1, 'Anjali Sharma');

SELECT * FROM Faculty;
```

Q27. Create an index on the Student(Email) column.

```
CREATE INDEX idx_student_email
ON Student(Email);
```

Q28. Create an index on the Course(CourseName) column for faster searching.

```
CREATE INDEX idx_course_name
ON Course(CourseName);
```

Q29. Create a composite index on Enrollment(StudentID, CourseID).

```
CREATE INDEX idx_enrollment_student_course
ON Enrollment(StudentID, CourseID);
```

SECTION 4: CREATE TABLES AND INSERT THE GIVEN DATA

Q30. Create the Department table and insert the given data.

```
CREATE TABLE Department (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(100) NOT NULL,
   Location VARCHAR(50)
);

INSERT INTO Department (DeptID, DeptName, Location)
VALUES
(1, 'Computer Science', 'Block A'),
(2, 'Mathematics', 'Block B'),
(3, 'Physics', 'Block C');

SELECT * FROM Department;
```

Q31. Create the Student table and insert the given data.

```
CREATE TABLE Student (
   StudentID INT PRIMARY KEY,
   Name VARCHAR(100) NOT NULL,
   Email VARCHAR(100) UNIQUE,
   Age INT CHECK (Age > 18),
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);

INSERT INTO Student (StudentID, Name, Email, Age, DeptID)
VALUES
(101, 'Rahul', 'rahul@example.com', 20, 1),
(102, 'Priya', 'priya@example.com', 21, 2),
(103, 'Amit', NULL, 19, 1),
(104, 'Sneha', 'sneha@example.com', 22, 3);

SELECT * FROM Student;
```

Q32. Create the Course table and insert the given data.

```
CREATE TABLE Course (
   CourseID INT PRIMARY KEY,
   CourseName VARCHAR(100),
   Credits INT CHECK (Credits BETWEEN 1 AND 6),
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);

INSERT INTO Course (CourseID, CourseName, Credits, DeptID)
VALUES
(201, 'Database Systems', 4, 1),
(202, 'Linear Algebra', 3, 2),
(203, 'Quantum Mechanics', 4, 3);

SELECT * FROM Course;
```

Q33. Create the Enrollment table and insert the given data.

```
CREATE TABLE Enrollment (
   EnrollID INT PRIMARY KEY,
   StudentID INT,
   CourseID INT,
   Grade CHAR(1),
   FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
   FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);

INSERT INTO Enrollment (EnrollID, StudentID, CourseID, Grade)
VALUES
(301, 101, 201, 'A'),
(302, 102, 202, 'B'),
(303, 103, 201, 'C'),
(304, 104, 203, 'A');

SELECT * FROM Enrollment;
```